

Introduction

In today's world, almost every app or system does many things at once. Whether it's a mobile app loading content while you're scrolling, or a backend server handling hundreds of users at the same time — everything is happening together, side by side.

But when many things run at once, they often bump into each other. Imagine a busy kitchen with several chefs working on the same dish without talking to each other. What happens? Confusion, mistakes, and maybe even a spoiled meal. The same kind of chaos can happen in software if things aren't managed properly.

This article is about that hidden chaos — and how we can bring order to it. We'll explore what really goes on behind the scenes when different parts of a program run at the same time, and how to make sure they don't get in each other's way.

Thread Safety

Thread safety means that a piece of code, object, or method works correctly and consistently when used by multiple threads at the same time. It makes sure that no wrong result is produced and no data gets corrupted — even if many threads are accessing or changing the same thing.

In simple words, thread safety ensures **everything stays in order**, even when many things are happening at once.

Real-life Analogy: TUF+ Purchase Counter

Imagine a TUF+ sale going live. Hundreds of users try to buy the premium version at the same time. There's a counter keeping track of how many purchases have happened.

If two users hit **"Buy"** at the same time, and the counter isn't protected, both may see the same count and increase it incorrectly — leading to messed-up records or over-selling.

Thread safety ensures that the **purchase counter is updated properly**, no matter how many users are trying to buy at once.

Why it Matters

Without thread safety, apps can behave unpredictably. Bugs appear that are hard to track, and things break randomly. That's why making code thread-safe is a must when multiple threads or users are involved.

Race Condition

When two or more tasks reach for the same data at the same moment, the *first* one to finish "wins," and the final result depends on sheer timing — not on logic. That timing lottery is called a **race condition**.

Real Life Analogy

Picture two cashiers writing today's sales total on a whiteboard. Each walks up, reads "₹ 10,000," adds "+₹1,000," and writes "₹11,000." They never saw the other cashier's update, so one sale is lost. The number you read depends on who grabbed the marker last.

Understanding

Let's understand this better with the help of a code example where **two threads are incrementing the same counter at the same time, racing to update the shared value without coordination.**

```
import java.util.concurrent.*;

// Purchase counter with no protection

class PurchaseCounter {

    // Shared count value

    private int count = 0;

    // Increment the counter

    public void increment() {

        // READ current value

        // INCREMENT it

        // WRITE it back

        count++;          // <-- not atomic, unsafe

    }

    // Fetch the current count

    public int getCount() {

        return count;

    }

}
```

```
}
```

```
// Demonstrates the race condition
```

```
class RaceConditionDemo {
```

```
    public static void main(String[] args) throws InterruptedException {
```

```
        // Create a shared counter
```

```
        PurchaseCounter counter = new PurchaseCounter();
```

```
        // Task that bumps the counter 1000 times
```

```
        Runnable task = () -> {
```

```
            for (int i = 0; i < 1000; i++) {
```

```
                counter.increment();
```

```
            }
```

```
        };
```

```
        // Run the same task in two threads
```

```
        Thread t1 = new Thread(task);
```

```
        Thread t2 = new Thread(task);
```

```
        t1.start();
```

```
        t2.start();
```

```
        t1.join();
```

```
t2.join();

// Expect 2000, but rarely get it

System.out.println("Final Count: " + counter.getCount());
}
}
```

When you run the above code, you won't always get the expected output. Ideally, the result should be **2000**, since both threads are supposed to increment the counter **1000** times each.

However, you'll often get a **smaller number** — because both threads are interfering with each other while updating the shared value.

Why the output is Inconsistent?

- `count++` looks like one step but is really three: **read, add 1, write back**.
- If both threads read the same value (say **42**) before either writes, each writes **43**. One increment vanishes.
- With many interleavings, the final count lands anywhere **below 2000**.

Wrap-up

Race conditions hide in plain sight and only appear under load — exactly when **reliability** matters most. In the next sections, we'll see how

synchronization tools (locks, atomic classes, etc.) eliminate these timing hazards while keeping our code fast and correct.

Synchronized Keyword

Why We Need It

We just saw how two threads can clash over the same data. To stop that clash, Java offers the **synchronized keyword** — a built-in way to let only **one** thread touch a critical section at a time.

How it Fixes Race Condition

When a thread enters a **synchronized** region, it grabs a **monitor lock** tied to an object (or the class itself).

- **If the lock is available:** the thread enters and safely runs the code inside.
- **If the lock is already held by another thread:** the thread waits (is blocked) until the lock is released.

This way, only **one thread at a time** can execute the synchronized part, which avoids conflicts and ensures thread safety.

Let us now understand the two common forms of using the **synchronized** keyword.

1. Synchronized Method

```
class SafeCounter {  
  
    private int count = 0;
```

// Entire method is protected by the instance's monitor lock

```
public synchronized void increment() {  
    count++;    // atomic under the lock  
}
```

```
public synchronized int getCount() {  
    return count;  
}  
}
```

- The method implicitly locks on **this** (the current object).
- Only one thread can run **increment()** (or any other **synchronized** method of the same object) at a time.

Drawback

Sometimes, using the **synchronized** keyword on an entire method can be **too restrictive**, especially when only a small part of the method actually deals with shared data.

In such cases, it's better to use a **synchronized block** to limit locking to just the critical section, improving both **performance and flexibility**.

2. Synchronized block

```
class SafeCounter {  
    private final Object lock = new Object();
```

```
private int count = 0;
```

```
public void increment() {
```

```
    // Lock only the code that truly needs protection
```

```
    synchronized (lock) {
```

```
        count++;
```

```
    }
```

```
}
```

```
public int getCount() {
```

```
    // No lock needed for simple read, or use block if strict consistency  
    required
```

```
    return count;
```

```
}
```

```
}
```

- You pick the **exact** section to lock, improving performance.
- To do this, you use a separate object (often named `lock`) just for synchronization. This object can be any normal Java object — commonly created once and reused inside the class to keep things organized and safe.

Using a **synchronized** block allows you to define exactly **which part of the code** should be protected, instead of locking the entire method.

This gives you the ability to place synchronization **only around the shared resource**, reducing unnecessary blocking and making your code more **efficient and maintainable** in the long run.

Monitor Locks

At the core of synchronization in Java lies the concept of **Monitor Locks**, which manage access by allowing only one thread to execute a synchronized section at any given time. Let's take a closer look at how Monitor Locks work through the given key points.

Key Points

- **One lock per object (plus one per Class for static sync).**
- **Re-entrant:** The same thread can enter the lock multiple times without deadlocking itself.
- **Queueing:** If a lock is taken, other threads wait (block) until it's released.
- **Scope matters:**
 - **synchronized instance method** → lock on **this**.
 - **synchronized static method** → lock on the **Class** object.
 - **synchronized (obj) block** → lock on **obj**.

Drawback

Using monitor locks via **synchronized** removes data races, but over-locking can **slow your app** or **cause deadlocks** if locks are acquired in inconsistent order.

While monitor locks help manage **exclusive access**, they also introduce overhead by forcing threads to wait, especially when only **visibility** of changes — not atomicity — is needed.

In such cases, where synchronization might be **too heavy**, Java provides a lighter option: the **volatile keyword**.

volatile Keyword

The **volatile** keyword in Java is used to ensure **visibility**, not atomicity. It tells the JVM that a variable's value may be updated by multiple threads and that **every read or write** should go directly to and from **main memory**, rather than being cached in a thread's local memory (CPU cache).

What it Guarantees:

1. visibility

Any update made to a **volatile** variable by one thread becomes **immediately visible** to all other threads.

- **Without volatile:** one thread might keep reading an old value from its local cache.
- **With volatile:** all threads will always see the latest value in memory.

```
class SharedData {  
  
    volatile boolean flag = false;  
  
    public void writer() {  
  
        flag = true;
```

```

    }

    public void reader() {
        if (flag) {
            // guaranteed to see true if another thread wrote it
        }
    }
}

```

2. No Caching

A **volatile** variable is always read from and written to main memory. This ensures there's **no outdated copy** sitting in a CPU register or thread-local cache.

3. Not Atomic

Even though **volatile** ensures visibility, it does **not make operations atomic**. For example, the following is **not thread-safe** even if **count** is declared as **volatile**:

```

class Counter {
    volatile int count = 0;

    public void increment() {
        count++; // Still unsafe!
    }
}

```

```
}
```

This is because `count++` involves **three steps** (read, modify, write), and `volatile` cannot stop two threads from performing these steps **simultaneously**, leading to race conditions.

When to Use `volatile`

`volatile` is best suited for scenarios where:

- One thread writes to a variable, and others only **read** it.
- There's **no need for atomic operations**, just **fresh visibility**.

For example: flags, status checks, or configuration values that might change during runtime.

Comparison Table: `volatile` vs `synchronized`

Feature	<code>volatile</code>	<code>synchronized</code>
Visibility	Ensures latest value is visible	Ensures visibility
Atomicity	Not guaranteed	Guaranteed for synchronized blocks
Lock Overhead	No locking, low cost	Involves locking, more overhead

Use Case	Simple flags, config variables	Shared counters, critical sections
Blocking	Non-blocking	Can block other threads

Atomic Variables

Java provides a set of classes under the `java.util.concurrent.atomic` package, designed to handle common types like integers and booleans in a thread-safe, high-performance way — without using locks.

Real-Life Analogy: Social Media Likes

Think of a "Like" counter on a popular Instagram post. Millions of users can tap the ❤️ at the same time. The system needs to **safely update the like count** without missing any, and it must do this without slowing everyone down with locks.

Atomic variables work the same way — offering **safe, fast updates** even under heavy traffic.

Common Atomic Classes

- `AtomicInteger` – for atomic operations on integers
- `AtomicBoolean` – for managing flags safely

(There are also `AtomicLong`, `AtomicReference`, etc., for more advanced use cases.)

How Atomic Variables Work

All atomic classes use a technique called **Compare-And-Swap (CAS)** at the hardware level. This is what makes them **lock-free** and **highly performant**.

CAS Concept

CAS stands for **Compare-And-Swap**. It is a low-level CPU/hardware instruction that checks if a memory location holds an expected value, and if so, swaps it with a new value — all in one atomic step. Here's how it works in simple terms:

“If the current value is what I expect it to be, update it with a new value. Otherwise, try again.”

This way, threads don't block each other—they simply keep retrying until they succeed, avoiding race conditions without using locks.

Example: Using `AtomicInteger` and `compareAndSet`

```
import java.util.concurrent.atomic.AtomicInteger;
```

```
class PurchaseAtomicCounter {
```

```
    // A thread-safe integer backed by hardware-level CAS
```

```
    private final AtomicInteger likes = new AtomicInteger(0);
```

```
    // Atomically add 1 to the like counter
```

```
    public void incrementLikes() {
```

```

int prev, next;

do {

    // Step 1 – read the current value.

    //      (May be outdated if another thread wins the race.)

    prev = likes.get();

    // Step 2 – compute the desired next value.

    next = prev + 1;

    // Step 3 – attempt to swap:

    /*      "If current value is still 'prev', set it to 'next'."
     *      Returns true on success, false if another thread
     *      already changed the value (retry needed).
     */

    } while (!likes.compareAndSet(prev, next));

}

// Read-only accessor

public int getCount() {

    return likes.get();
}

```

```
}  
  
}
```

Why `AtomicInteger`?

- Holds a single `int` that can be **updated without locks**.
- Internally uses the CPU's **Compare-And-Swap (CAS)** instruction, giving high throughput under moderate contention.

What is `compareAndSet(old, new)` ?

- **Compare** the variable's current value to `old`.
- If **equal**, swap in `new` atomically and return `true`.
- If not, do **nothing** and return `false` (someone else changed it first).

How the Loop Works

The table below assumes both threads (U1 and U2) are trying to increment the same atomic counter simultaneously. The operations are broken down step by step to show how `compareAndSet()` ensures safe updates using **Compare-And-Swap (CAS)** internally.

Iteration Step	Thread U1 Value Flow	Thread U2 Value Flow	Outcome
<code>prev = get()</code>	reads 10	reads 10	both see same start
<code>next = prev + 1</code>	computes 11	computes 11	both want 11

<code>compareAndSet()</code>	U1 succeeds (10 → 11)	U2 fails (value now 11)	U1 wins
Retry loop	exits	retries with new <code>prev=11, next=12</code>	eventually U2 sets 12

Compare-And-Set vs Compare-And-Swap

They refer to the **same underlying concept**, but in different contexts:

Term	Meaning
Compare-And-Swap (CAS)	A low-level CPU/hardware instruction that checks if a memory location holds an expected value, and if so, swaps it with a new value — all in one atomic step.
Compare-And-Set	A Java-level method (like <code>AtomicInteger.compareAndSet()</code>) that uses the hardware CAS under the hood to implement safe, lock-free updates.

Pros

- **High performance:** No locking or blocking
- **Simple API:** Methods like `get()`, `set()`, `incrementAndGet()`, `compareAndSet()`

Cons

- May **fail under high contention** (many threads retrying at once)
- Only works well for **single variables** (not ideal for compound or group updates)

Synchronized vs Volatile vs Atomic Variables

Different concurrency tools serve different purposes. Here's a quick comparison of **synchronized**, **volatile**, and **AtomicInteger** to understand when and why each one should be used.

Feature	synchronized	volatile	AtomicInteger
Guarantees Atomicity	✓	✗	✓
Guarantees Visibility	✓	✓	✓
Blocking	✓	✗	✗
Performance	Lower	High	High
Use Case	Complex operations	One writer, many readers	Simple counters or flags